



Introduction

- Find a code base
- Perform Static and Dynamic analysis on that.
- Static Analysis
 - Generate AST
 - Metrics Calculation
- Dynamic Analysis
 - Code coverage
- Find a suitable tools to support your analysis
- Prepare a report on your work providing necessary screenshots and explanation



Find the Codebase

<https://github.com/avinash201199/Projects>

Dynamic Analysis and Static Analysis

Dynamic Analysis



Def: Dynamic analysis is the ability to track what is happening during program execution.

The Python `settrace()` mechanism allows you to track all code lines, all variables, all values, as the program executes

Python `sys` module provides some of the powerful functions but they are complex to understand. One of which is the `sys.settrace()` which is used for implementing debuggers, profilers and coverage tools.

On a higher level, `sys.settrace()` registers the traceback to the Python interpreter. A traceback is basically the information that is returned when an event happens in the code.

Dynamic Analysis



The registered traceback is invoked when one of the following four events occur :

1. Function is called
2. Function returns
3. Execution of a line
4. Exception is raised

Dynamic Analysis



Syntax: `sys.settrace(frame, event, arg.frame)`

Parameters:

frame: frame is the current stack frame

event: A string which be either 'call', 'line', 'return', 'exception' or 'opcode'

arg: Depends on the event type

Returns: Reference to the local trace function which then returns reference to itself

Example 1:



```
import sys
def my_tracer(frame, event, arg = None):
    code = frame.f_code
    func_name = code.co_name
    line_no = frame.f_lineno
    print(f"A {event} encountered in \
{func_name}() at line number {line_no} ")
return my_tracer
```

Example 1:



```
def fun(a, b):  
    c=a+b  
    return c
```

```
def check():  
    return fun(1,2)  
sys.settrace(my_tracer)  
print(check())  
sys.settrace(None)
```



Output of Example 1

```
>>> runfile('/root/PycharmProjects/untitled2/test.py', wdir='/root/PycharmProjects/untitled2')
A call encountered in      check() at line number 13
A line encountered in      check() at line number 14
A call encountered in      fun() at line number 10
A line encountered in      fun() at line number 11
A line encountered in      fun() at line number 12
A return encountered in     fun() at line number 12
A return encountered in     check() at line number 14
3
```

Example 2:



```
import sys
import inspect
def my_tracer(frame, event, arg):
    function_code = frame.f_code
    function_name = function_code.co_name
    lineno = frame.f_lineno
    vars = frame.f_locals
    source_lines, starting_line_no = inspect.getsourcelines(frame.f_code)
    loc = f"{function_name}:{lineno} {source_lines[lineno -
starting_line_no].rstrip()}"
    vars = ", ".join(f"{name} = {vars[name]}" for name in vars)
    print(f"{loc:50} ({vars})")
    return my_tracer
```



Dynamic Analysis (Scalene Tool)

The Memory Profiler is a component that **helps you identify memory leaks and memory churn** that can lead to stutter, freezes, and even app crashes.

Installation: Pip Install Scalene

Github Link: <https://github.com/plasma-umass/scalene>



Static Analysis

Static analysis refers to the ability to analyze *program code* without actually executing it.

An Abstract Syntax Tree (AST) is a tree representation of the syntactic structure of source code, where: Nodes represent constructs in the source code (e.g., variables, loops, functions). Branches represent relationships between those constructs (e.g., "a loop contains a condition").



Generate AST

- <https://dev.to/balapriya/abstract-syntax-tree-ast-explained-in-plain-english-1h38>
- AST Explorer: <https://astexplorer.net/>

Example 1:

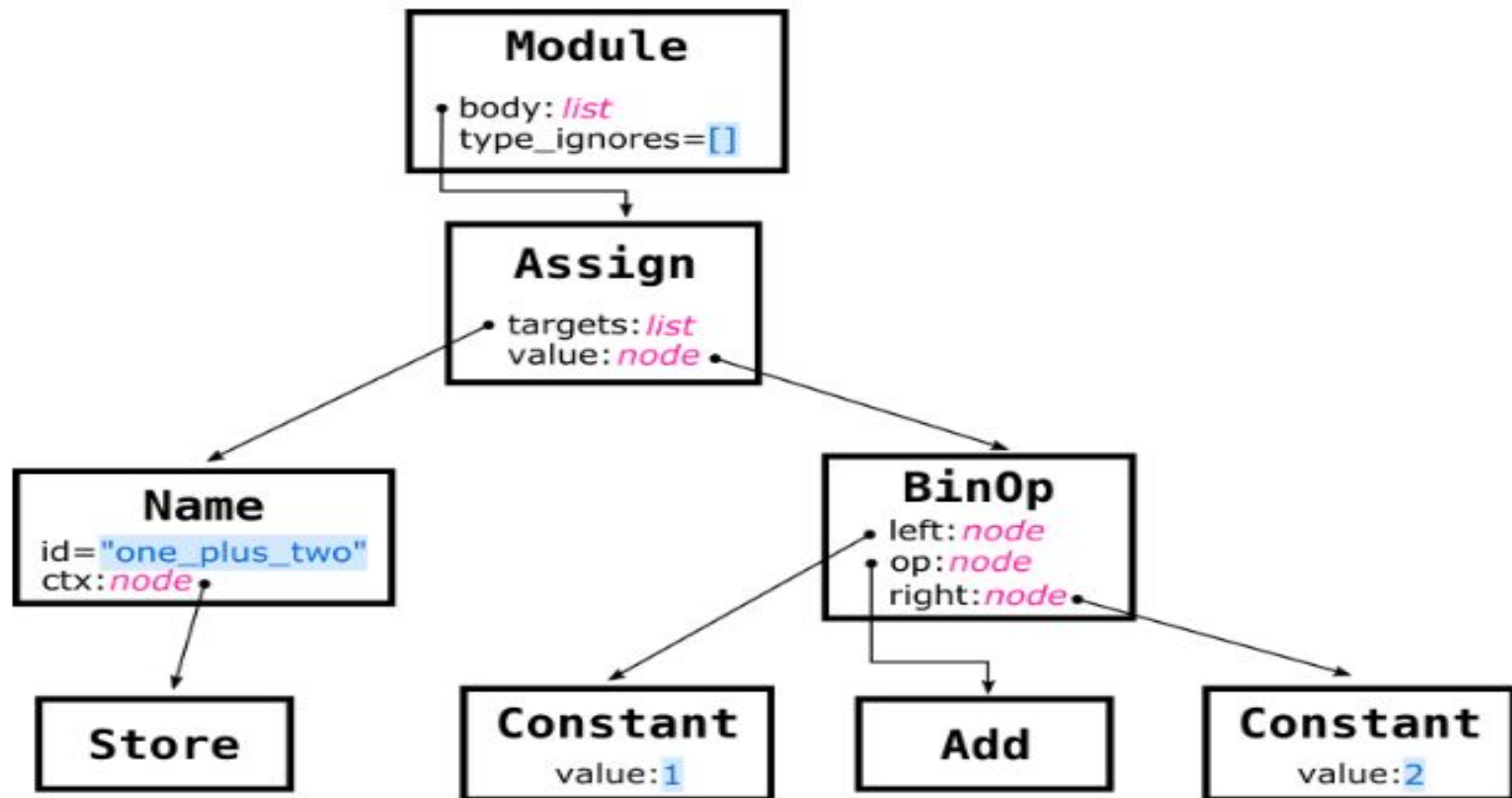


```
import ast
```

```
code = "one_plus_two = 1+2"
```

```
tree = ast.parse(code)
```

```
print(ast.dump(tree))
```



Example 2:



```
import ast
tree_ast = ast.parse("""
subjects = ['computer science', 'alorithm']

name = 'Ricky'
for sub in subjects:

    print('{} learn {}'.format(name, sub))

""")
print(ast.dump(tree_ast))
```



Sonarqube Static Analysis tool

<https://www.sonarsource.com/products/sonarqube/>